

Process Intelligence with Python Language

1.5M

events in BPI 2019 dataset

21

Python tools reviewed

1.000

conformance fitness achieved

Author Thi Van Trang Le - ID 140036

Supervisor dr hab. Michal Ramsza - Institute of Mathematical Economics

Program Advanced Analysis - Big Data

CHAPTER 1 - INTRODUCTION

Research Questions & Objectives

“Which Python tools are available for process mining and process intelligence, and how well do they support process discovery, conformance, and performance analysis?”

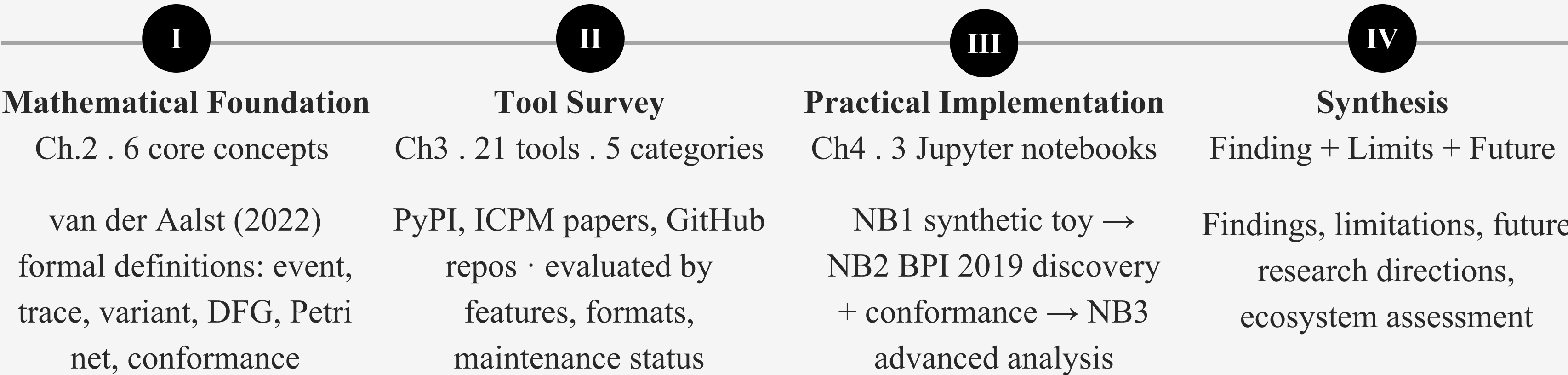
Context: PM4Py was introduced in 2018. Python ecosystem for process mining remains fragmented, inconsistently documented, and underexplored in academic literature.

Scope: Open-source Python tools only · 3 core tasks (discovery, conformance, performance) · BPI Challenge 2019 as primary case study · 3 reproducible Jupyter notebooks

4 OBJECTIVES

- **Formalize** mathematical foundations of process mining - events, traces, Petri nets, conformance
- **Survey** 21 Python libraries across 5 functional categories
- **Demonstrate** full pipeline on BPI 2019 with reproducible notebooks
- **Conclude** on ecosystem maturity and identify research gaps

Four Sequential Phases



***Important Note:** Methodology is descriptive and demonstrative, not a formal benchmark. No runtime or accuracy comparisons between competing algorithms are performed.*

Events, Traces, Variants & Logs

$$e = (c, a, t)$$

Event

Case ID · activity label · timestamp. In practice also: resource, cost, document type, org. unit. Minimum: c + a + t required.

$$\sigma = \langle a_1, a_2, \dots, a_n \rangle$$

Trace

Ordered sequence of activity labels for one case. Length $|\sigma| = n$. Each case in the log maps to exactly one trace.

$$L \in \mathcal{B}(A^*)$$

Event Log

Multiset of traces over activity alphabet A^* . $L(\sigma)$ = number of cases following σ . BPI 2019 (flattened): L has 43,898 traces.

$$V(L) = \{\sigma \mid L(\sigma) > 0\}$$

Variant

Unique trace pattern shared by ≥ 1 cases. BPI 2019: 776 distinct variants for 43,898 cases $\rightarrow$ process is structured but not trivial.

$$DFG(L) = (A_L, \rightarrow_L, f_L)$$

Directly-Follows Graph

Nodes = activities · Directed edges = observed transitions · $f_L(a \rightarrow b)$ = frequency of a immediately followed by b.

$$TT(c) = \max t(e) - \min t(e)$$

Throughput Time

First to last event for one case. BPI 2019: median 17.89 days · mean 36.67 days · P95 125.98 days.

Petri Nets, Soundness & Conformance

$N = (P, T, F, W, M_0)$

Petri Net

P = places (states/conditions)

T = transitions (activities)

$F \subseteq (P \times T) \cup (T \times P)$ = flow relation

W = arc weights

M_0 = initial marking (token distribution)

$\text{fitness} = \frac{1}{2}(1 - m/c) + \frac{1}{2}(1 - r/p)$

Token-Based Replay Fitness

m = missing tokens (model can't fire, adds artificially)

c = consumed tokens

r = remaining tokens at end

p = produced tokens

fitness = 1.000 means every trace replays perfectly — no missing, no remaining tokens.

Sound $\Leftrightarrow$ ①②③

Soundness — 3 Conditions

→ ① Reachability — final marking always reachable

→ ② Option to complete — no forced dead states

→ ③ Proper termination — no extra tokens at end

IMf (Inductive Miner Infrequent) **guarantees a sound net** by design.

OCEL: $e \rightarrow \{o_1, o_2, \dots, o_k\}$

Object-Centric Event Log

Standard XES: 1 event $\rightarrow$ 1 case. OCEL: 1 event $\rightarrow$ multiple objects simultaneously. BPI 2019: 53,198 events linked to 71,559 objects across 4 types.

21 Python Tools across 5 Categories

CORE PM	PREDICTIVE / AI	SIMULATION	VISUALIZATION	DATA PROCESSING
PM4Py 2.7.19 ★	ML4ProM	SimPy	Graphviz	pandas
PM4Py-Streaming	PM4Py-ML	BPSim	Plotly / Dash	NumPy
Simod	PyCelonis \$\$	ProcessOpt.	NetworkX	scikit-learn
PMLab	bpmn-python		Pyvis	XGBoost
BupaR (R port)	PPMT			TF / PyTorch

Why PM4Py? Only open-source library supporting the **complete workflow**: OCEL loading → flattening → DFG → Petri net discovery → token-based replay → performance DFG → OCDFG → predictive monitoring.
Actively maintained. Native Python API.

From OCEL Load to Predictive Output

```
# 1. Load OCEL (Object-Centric Event Log) ocel =  
pm4py.read_ocel("BPIC19.jsonocel") # 2. Flatten to flat log by object type  
log = pm4py.ocel_flattening(ocel, "POItem") # 3. Discover Petri net —  
Inductive Miner Infrequent net, im, fm = pm4py.discover_petri_net_inductive(  
log, noise_threshold=0.2) # 4. Conformance — token-based replay  
result = pm4py.fitness_token_based_replay(log, net, im, fm)  
#result['average_trace_fitness'] → 1.0 # 5. Performance DFG  
perf_dfg, sa, ea = pm4py.discover_performance_dfg(log)  
# 6. Object-Centric DFG  
ocdfg = pm4py.discover_ocdfg(ocel)
```

Six core PM4Py calls cover the entire analytical pipeline.
All used in Notebooks 2 and 3 on the BPI 2019 dataset.

ocel_flattening

OCEL → XES flat log

discover_petri_net_inductive

IMf discovery

**fitness_token_based_
replay**

conformance score

discover_performance_dfg

bottleneck timing

noise_threshold = 0.2 — IMf parameter that filters
variants covering <20% of cases, keeping the model
compact while guaranteeing soundness.

Procurement Process at a Dutch Manufacturing Company

Real ERP log from SAP system. Covers purchase-order process: creation, approval, goods receipt, invoice verification. Exported as OCEL (Object-Centric Event Log).

53,198

total events

71,559

total objects

4

object types

43,898

cases after flatten

4 Object Types

- **PurchaseOrder** — the PO header document
- **POItem** — individual line items [flattening key]
- **Resource** — users who performed events
- **Vendor** — external supplier

OCEL vs. XES comparison

Why OCEL format?

Standard XES: event belongs to exactly 1 case. OCEL: 1 event can simultaneously belong to a PurchaseOrder, a POItem, and a Vendor. This reflects real process complexity where one invoice touches multiple PO lines.

Pre-processing applied

Data Cleaning Steps

- Removed events with timestamps before **2018-01-01** (data entry errors)
- Flattened by **POItem** → one case per line item
- Verified no duplicate events per case

Result after cleaning: **43,898 cases** used for discovery, conformance, and advanced analysis.

Chronological 70/15/15 Split

Why chronological, not random?

Event logs are time-ordered. A random split would place future events in the training set and past events in the test set — this is **data leakage**: the model would implicitly learn from information not available at prediction time.

- ① **Sort** all 43,898 cases by first-event timestamp ascending
- ② **Train** — first 70% = **30,726 cases** · used for IMf discovery
- ③ **Validation** — next 15% = **6,584 cases** · conformance check + hyperparameter tuning
- ④ **Test** — final 15% = **6,586 cases** · held out until final evaluation
- ⑤ **No overlap** — each case appears in exactly one split

30,726

training cases (70%)

6,584

validation cases (15%)

6,586

test cases (15%)

43,898

total . no case dropped

IMf trained on 30,726 cases only

Why train on 70%?

Process model must be discovered without seeing future behaviour. The Petri net discovered on training data is then evaluated on validation and test sets — a genuine out-of-sample assessment.

Proof of Concept on a Hand-Crafted Log

Purpose: validate PM4Py functions in a **controlled setting** before scaling to real data. Three cases, one process path, results verifiable by hand.

Process: Create PO → Approve PO → Send Invoice

```
import pm4py, pandas as pd # Hand-crafted: 3 cases × 3 activities
df = pd.DataFrame({
    "case:concept:name": ["C1","C1","C1","C2",...],
    "concept:name": ["Create PO","Approve",...],
    "time:timestamp":
        pd.to_datetime([...]) }) log = pm4py.convert_to_event_log(df) net, im, fm =
pm4py.discover_petri_net_inductive(log)
replayed = pm4py.fitness_token_based_replay(log, net, im, fm)
tt = df.groupby("case:concept:name")["time:timestamp"].agg(["max","min"])
```

Output figures: fig_synth_dfg.png · fig_synth_petri_net.png ·
fig_synth_waiting_times.png

3 cases	3 activities
1 unique variant	4 Petri net places
3 transitions	6 arcs
1.000 fitness · mean throughput 120 min · CV = 0.04	

Key insight: Simple 1-variant log → compact sound Petri net → perfect fitness. Confirms PM4Py pipeline works correctly before scaling to 43,898 cases.

Inductive Miner Infrequent on BPI 2019

IMf is chosen over Alpha Miner and Heuristics Miner because it **guarantees a sound Petri net** — no deadlocks, no livelocks — while still handling noise through the threshold parameter.

- ① **Input:** 30,726 training cases · flat XES log
- ② **IMf discovers process tree** first (recursive divide-and-conquer), then converts to Petri net **noise_threshold=0.2** filters rare variants
- ③ **Output:** (net, im, fm) — Petri net + initial marking + final marking
- ④ **Soundness verified** by construction — IMf guarantees no deadlock

Alpha Miner vs. IMf

Alpha Miner is simpler but fails on loops and long dependencies. IMf handles both and guarantees soundness. **Trade-off:** IMf model is more complex (95 places) but more accurate.

DISCOVERY PETRI NET - BPI 2019

95
places (P)

154
transitions (T)

334
arcs (F)

30,726
training cases

Sound Petri net — guaranteed by IMf — no deadlocks, no livelocks

The model complexity (95P + 154T) reflects the real procurement process: parallel paths for 1-way vs. 3-way matching, optional approval steps, and document type variants.

Token-Based Replay on Held-Out Cases

$$\text{fitness} = \frac{1}{2}(1 - m/c) + \frac{1}{2}(1 - r/p)$$

Token-Based Replay Algorithm

For each trace σ , simulate token flow through Petri net: fire transitions in order. If transition can't fire $\rightarrow$ add missing token ($m++$). At end $\rightarrow$ count remaining tokens (r). Aggregate over all cases.

① **Replay each case** on the Petri net discovered from training data

② **Count per trace:** missing (m), consumed (c), remaining (r), produced (p)

③ **Aggregate** across all cases $\rightarrow$ average_trace_fitness

④ **Fitness = 1.000** $\rightarrow$ all 6,584 + 6,586 cases replay perfectly

1.000

fitness — validation
(6,584 cases)

1.000

fitness — test
(6,586 cases)

100%

perfectly fitting traces

0

missing tokens across
all cases

$m = 0 \cdot r = 0$ for every case $\rightarrow$ model is a complete explanation of log

Interpretation: IMf discovered a sound Petri net from 30,726 training cases that **perfectly explains all 13,170 held-out cases**. The BPI 2019 procurement process is highly structured — very few genuinely unexpected deviations.

Variant Analysis & Process Concentration

A variant is a unique sequence of activities. Analysis reveals how concentrated or dispersed the process actually is across 43,898 cases.

776
distinct variants

43,898
total cases

85.3%
covered by top-10
variants

90.5%
covered by top-20
variants

**Remaining 756 variants cover only 9.5%
→ long tail of rare deviations**

TOP VARIANTS BY COVERAGE

V1 — Create PO Item → Record GR → Record Invoice **~28%**

V2 — + Approval step inserted **~18%**

V3 — 3-way match path (GR + service entry) **~12%**

V4–10 — Variations on approval and matching **+27.3%**

V11+ — 756 rare variants — corrections, exceptions **9.5%**

Implication: The process is concentrated — a small number of standard paths cover most activity. This justifies using IMf with noise filtering: rare paths are genuinely rare, not common process variants.

Rework Detection & Throughput Time

REWORK - REPEATED ACTIVITIES

17.89d

median throughput time

125.98d

P95 throughput time

Most common rework: **"Record Service Entry Sheet"**

Suggests manual correction cycles — an activity that requires re-entry indicates upstream errors (wrong quantity, wrong vendor). Rework cases have significantly longer throughput time than non-rework cases.

PERFORMANCE DFG - BOTTLENECK

Largest time gap: **goods receipt** → **invoice verification**. Median waiting time between these steps is significantly higher than all other transitions — primary bottleneck in the procurement process.

THROUGHPUT TIME DISTRIBUTION

17.89d

median throughput
time

36.67d

mean throughput time

125.98d

P95 throughput time

4.9%

rework rate

mean / median ratio = 2.05

→ strongly right-skewed distribution

Key insight: Mean (36.67d) is more than double the median (17.89d). A small number of slow outlier cases inflates the mean. For SLA reporting, **median is the appropriate KPI**, not mean.

Performance DFG & Automation Rate

PERFORMANCE DFG — SLOWEST TRANSITIONS

Performance DFG annotates each arc with **median waiting time** between activities.

Discovered via `pm4py.discover_performance_dfg(train_log)`

Rank 1–3 — **Vendor creates invoice** → **Clear Invoice** — longest median delay invoice clearing is the primary bottleneck in payment cycle

Rank 4–6 — **Record GR** → **Record Invoice Receipt** — goods receipt to invoice gap waiting for 3-way matching verification

Rank 7–10 — **Approval-related transitions** — depends on approver availability manual steps = resource bottleneck risk

`perf_dfg[arc]["median"] / 86400`

Times stored in seconds → converted to days. Median used (not mean) because throughput distributions are right-skewed — same insight as overall throughput analysis.

AUTOMATION RATE PER ACTIVITY

Resources classified by name: **batch*** → **automated**, others → manual. Activities with >50% automated events highlighted in blue.

batch*

resource name pattern
= automated

human

named user = manual
activity

50%

threshold line on chart

fig_08

automation_rate.png

Key insight: Activities near payment (Clear Invoice, Remove Payment Block) tend to be highly automated. Activities near goods receipt and service entry tend to be manual — these are also where rework is most common.

Object-Centric Directly-Follows Graph (OCDFG)

PERFORMANCE DFG — SLOWEST TRANSITIONS

Standard DFG collapses all object types into one flat view. OCDFG computes a separate DFG per object type, revealing how the same events look from different object perspectives.

OCDFG: one DFG per object type

Why OCDFG matters

In BPI 2019: one invoice event links to both a PurchaseOrder and multiple POItems. Standard DFG shows one combined view. OCDFG shows: how does the process look if we follow PO objects? And separately: how does it look following POItem objects?

① **Extract relations** from OCEL — ocel.relations contains event↔object links

② **Filter by object type** — process each type (POItem, PO) separately

③ **Compute DFG per type** — group by ocel:oid, sort by timestamp, shift(-1) for next activity

④ **Compare** — same activities, different transition frequencies and paths

POITEM VS PO PERSPECTIVE

POItem

43,903 objects · line
item level · most

PO

25,825 objects · header
level · aggregates items

fig_09_ocdfg.png

Two side-by-side bar charts — top 10 directly-follows pairs per object type. POItem shows fine-grained item lifecycle. PO shows header-level document flow. Different dominant paths emerge for each perspective.

Insight: Flattening (used in NB2) forces one perspective. OCDFG reveals the **multi-dimensional nature** of the process — a limitation of classical process mining that OCEL-native tools are designed to overcome.

Simulation & Predictive Monitoring

SIMPY PROCESS SIMULATION

SimPy discrete-event simulation models the approval queue. Service times taken from performance DFG median values. Tests 1–6 approvers over a 365-day horizon.

- ① **Arrival rate:** exponential, mean inter-arrival = 0.5 days
- ② **Service time:** $N(\mu, 0.3\mu)$ where μ = mean of performance DFG medians
- ③ **Resource pool:** n approvers (capacity = n) — tested n = 1...6
- ④ **KPIs tracked:** mean, median, P95 throughput time per scenario

BPI 2019 mean $\approx$ 36.8 days (reference)

fig_10 shows: throughput drops sharply from 1→3 approvers, then diminishing returns. Adding a 4th approver beyond 3 improves less than going from 1→2.

PREDICTIVE MONITORING - RANDOM FOREST

Classify cases as "slow" (TT > 30 days) using only first-event attributes — information known at case start.

RF 100
Random Forest, 100 trees

70/30
stratified train/test split

Features: `cDocType` · `cGR` · `cGRbasedInvVerif` · `cCompany` · `cItemCat` · `cItemType`
Label-encoded. All available at order creation.
cDocType and **cGR** are top predictors — document type determines approval complexity, GR flag determines 3-way matching requirement.

Output: AUC-ROC score + feature importance chart (fig_11). Demonstrates that case outcomes can be predicted at creation time using only document metadata — no process history required.

Findings, Limitations & Future Work

Main Findings — RQ Answered

- Python + PM4Py supports **complete end-to-end** process mining workflow
- OCEL load → discovery → conformance → performance → prediction all in one ecosystem
- **Fitness 1.000** on 43,898-case real procurement log
- 100% reproducible via 3 public Jupyter notebooks
- Python ecosystem is **sufficient but less mature** than Java ProM

Limitations

- No formal **benchmarking** between algorithms (Alpha vs. IMf)
- Used **sample** (43,898 cases), not full BPI 2019 (~1.6M events) → OCEL flattening can introduce **case count artifacts**
- Conformance via token replay only — not alignment-based
- Not designed for **production / real-time** deployment

Future Research Directions

- Object-centric Petri nets with OCEL-native conformance checking → LSTM / Transformer for predictive process monitoring
- Alignment-based conformance on full 1.6M-event BPI 2019 dataset → Standardized Python PM benchmarking framework (runtime + quality)

1.5M	21	1.000	3
events in BPI 2019	tools reviewed	conformance fitness	reproducible notebooks